Year Up United
Data Analyst Training Academy: Continuing SQL

Week 3 Student Guide

This courseware is to be distributed to accompany instructor led training.

Copyright © 2026 Pluralsight & The Reliable Collaboration Company

Table of Contents

MODULE 1 QUERYING DATA 5

SECTION 1.A WRITING QUERIES 6

1.A.1 Week 3 Learning Objectives 7
1.A.2 Week 3 Learning Outcomes 8
1.A.3 What is a query? 9
1.A.4 `SELECT` Statements 10
1.A.5 Using Aliases 11
1.A.6 `WHERE` Clauses 12
1.A.7 `SELECT` … `WHERE` Statements 13

SECTION 1.B WORKING WITH CONDITIONAL LOGIC 14

1.B.1 Boolean logical operations 15
1.B.2 Comparison Operators & Functions 16
1.B.3 Logical Operators 17
1.B.4 Using `WHERE` with Multiple Conditions 18
1.B.5 Using the `NOT` Operator 19
1.B.6 Using the `IN` Operator 20
1.B.7 Using the `BETWEEN` Operator 21
1.B.8 Testing for `NULL` 22
1.B.9 Using the `LIKE` Operator 23
1.B.10 Single Quotes & Escape Characters 24
1.B.11 Performing a Date Search 25

SECTION 1.C OTHER QUERY CONCEPTS 26

1.C.1 Ordering Results 27
1.C.2 Sampling Records with `LIMIT` 28
1.C.3 Non-unique records with `DISTINCT` 29
1.C.4 Concatenation 30
1.C.5 Adding Calculations to Results 31

MODULE 2 QUERYING MULTIPLE TABLES 32

SECTION 2.A JOINING TABLES 33

2.A.1 Introducing Joins 34
2.A.2 Using Qualified Column Names 35
2.A.3 The `USING` Keyword 36
2.A.4 Inner Joins 37
2.A.5 Outer Joins 38
2.A.6 Types of Outer Joins 39
2.A.7 Which Join to Use? 40
2.A.8 Multi-table Joins 41
2.A.9 Adding `WHERE` Clauses to Joins 42
2.A.10 Using Self-Joins 43

SECTION 2.B CREATING AND USING VIEWS 44

2.B.1 Views 45
2.B.2 Creating a View 46
2.B.3 Modifying or Deleting a View 47

MODULE 3 WRITING COMPLEX QUERIES 48

SECTION 3.A USING AGGREGATE FUNCTIONS 49
3.A.1 Aggregate Functions 50
3.A.2 Tips when using aggregate functions 51
3.A.3 Grouping Results 52
3.A.4 `HAVING` Filter for Aggregated Columns 53
SECTION 3.B SCALAR FUNCTIONS 54
3.B.1 Overview of Scalar Functions 55
3.B.2 Data Type Conversion Functions 56
3.B.3 String Functions 57
3.B.4 Date Functions 58
3.B.5 NULL-related functions 59
3.B.6 Numeric/Math Functions 60
SECTION 3.C SET OPERATIONS 61
3.C.1 What Are Set Operations? 62
3.C.2 `UNION` Operator 63
3.C.3 Using `INTERSECT` 64
3.C.4 Using `EXCEPT` 65

MODULE 4 USING SUBQUERIES 66

SECTION 4.A WORKING WITH SUBQUERIES 67
4.A.1 Overview of Subqueries 68
4.A.2 Subqueries as Column Expressions 69
4.A.3 Subqueries in `FROM` or `JOIN` Clauses 70
4.A.4 Nesting Subqueries 71
4.A.5 Subqueries in Other Statements 72
4.A.6 Subqueries in Other Statements (cont.) 73
4.A.7 Types of Subqueries 74
4.A.8 Comparing Subqueries and Joins 75
4.A.9 Testing Set Membership with `IN` 76
4.A.10 Testing Existence with `EXISTS` 77
SECTION 4.B COMMON TABLE EXPRESSIONS 78
4.B.1 Common Table Expressions (CTEs) 79
4.B.2 More on CTEs 80
SECTION 4.C MORE QUERY OPTIONS 81
4.C.1 `CREATE TABLE … AS SELECT` 82
4.C.2 `INSERT INTO … SELECT` 83
4.C.3 `TRUNCATE TABLE` 84
SECTION 4.D TESTING FOR EFFICIENCY 85
4.D.1 Testing for Efficiency 86
4.D.2 Performance Tools in MySQL Workbench 87
4.D.3 Querying Query Efficiency 89
SECTION 4.E WORKING WITH PROGRAMMING LOGIC 90
4.E.1 Programming Logic Concepts 91
4.E.2 Variables 92
4.E.3 Declaring and Assigning Variables 93
4.E.4 Control flow tools 94

4.E.5 `CASE` Structures *95*
4.E.6 Searched `CASE` *96*
4.E.7 Overview of Conditional Execution *97*
4.E.8 `IF…ELSE` Syntax Variations *98*
4.E.9 Repeated Execution with Loops *99*

MODULE 5 OTHER DATABASE FEATURES 100

SECTION 5.A DATABASE INDEXES 101
5.A.1 What is an Index? *102*
5.A.2 Choosing which Columns to Index *103*
5.A.3 Problems with Indexing *104*
5.A.4 Adding an Index *105*
5.A.5 Other Types of Indexes *106*
5.A.6 Dropping an Index *107*
SECTION 5.B MORE CONCEPTS 108
5.B.1 Additional Database Concepts *109*
5.B.2 Types of Relational Databases *110*
5.B.3 Many DBMS Options Exist *111*
5.B.4 Microsoft SQL Server & SSMS *112*
5.B.5 Azure Data Studio *113*
5.B.6 3rd Party Client Software: DBeaver *114*
SECTION 5.C WRAP-UP 115
5.C.1 Review: SQL Fundamentals *116*
5.C.2 Best Practices *117*

Module 1

Querying Data

Section 1.A

Writing Queries

1.A.1 Week 3 Learning Objectives

- This week you will learn:
  - ◊ Writing SELECT queries with filtering, sorting, and conditional logic
  - ◊ Query result modifiers: aliases, ORDER BY, LIMIT, DISTINCT, and calculated columns
  - ◊ Joining multiple tables to query across related data
  - ◊ Creating and using views to encapsulate reusable queries
  - ◊ Aggregate functions and grouping results with GROUP BY and HAVING
  - ◊ Scalar functions for working with strings, dates, numbers, and NULL values
  - ◊ Set operations: UNION, INTERSECT, and EXCEPT
  - ◊ Subqueries: how and when to use them in SELECT, FROM, WHERE, and other clauses
  - ◊ Common Table Expressions (CTEs) as an alternative to subqueries
  - ◊ Query performance: how to evaluate and interpret query efficiency
  - ◊ Programming logic in SQL: variables, CASE expressions, IF/ELSE, and loops
  - ◊ Database indexes: what they are, when to use them, and the trade-offs involved
  - ◊ A brief overview of some other relational database platforms and tools beyond MySQL

1.A.2 Week 3 Learning Outcomes

- By the end of this week, you will be able to:
  - ◊ Write SELECT queries with WHERE clauses using comparison operators, logical operators, IN, BETWEEN, LIKE, IS NULL, and date filters
  - ◊ Sort, limit, and de-duplicate query results using ORDER BY, LIMIT, and DISTINCT
  - ◊ Add calculated columns to a query's result set and use column aliases
  - ◊ Join two or more tables using INNER JOIN and OUTER JOIN (LEFT, RIGHT) and explain when to use each
  - ◊ Write queries that aggregate data using COUNT, SUM, AVG, MIN, and MAX, grouped with GROUP BY and filtered with HAVING
  - ◊ Work with scalar functions to transform string, date, and numeric values within a query
  - ◊ Combine result sets from multiple queries using UNION, INTERSECT, and EXCEPT
  - ◊ Write subqueries in SELECT, FROM, WHERE, and JOIN clauses, and choose between IN and EXISTS appropriately
  - ◊ Write a CASE expression to apply conditional logic within a query
  - ◊ Describe what a database index is, identify good candidates for indexing, and explain the potential downsides

1.A.3 What is a query?

- In common usage (not specific to programming), a *query* is basically a question or an inquiry, and *querying* is asking/inquiring

- When it comes to interacting with relational databases, a **query** is any set of SQL commands to interact with, create, or seeks answers from the database (i.e. any request sent to the DB server)

- Last week we covered the concepts of DDL, DML, and DCL
  - ◊ DDL (Data Definition Language): create or modify database objects using CREATE, ALTER, or DROP
  - ◊ DML (Data Manipulation Language): insert or update data in the database using INSERT, UPDATE, DELETE, or SELECT
  - ◊ DCL (Data Control Language): sets permissions on database objects using GRANT or REVOKE

- All of the above are types of queries!

1.A.4 `SELECT` Statements

- The **`SELECT`** statement is used to **select** / retrieve data from a table in a database
  - ◊ Basic syntax:

    ```
    SELECT what FROM table;
    ```

    - * `SELECT *` retrieves all columns
    - * To only retrieve certain columns, specify by column name as `SELECT column1, column2` [etc.]
  - ◊ Optional clauses include `WHERE` or `JOIN` (we'll talk about those more...)

- `SELECT` can also be used to return the result of an expression
  - ◊ Example:

    ```
    SELECT 85 + 33;
    SELECT 'Do not delete';
    ```

1.A.5 Using Aliases

- Aliases are a way to provide a temporary name for a table, or a column from a table
  - ◊ Often aliases are used to make table or column names more reader-friendly
  - ◊ An alias may also be used to make a very long name shorter to type!
- An alias is created using `SELECT` … `AS` plus the alias
  - ◊ Syntax:

```
SELECT column_name AS col_alias
FROM table;
```

    - ∗ Note that putting a line break before `FROM` has no effect on the functionality of the query – it just makes the query easier to read
  - ◊ In most versions of SQL (including MySQL) the `AS` keyword is optional, so the same result can be achieved using:

```
SELECT column_name col_alias
FROM table;
```

1.A.6 `WHERE` Clauses

- The `WHERE` clause specifies criteria / condition that records must meet to be affected by the query
  - ◊ Used with `SELECT` to specify criteria that records must match in order to be returned
  - ◊ Used with `UPDATE` and `DELETE` statements to indicate which records to modify or delete

- `WHERE` requires three arguments:
  - ◊ The name of the column to be matched against
  - ◊ A comparison operator (`=, <, <=, >, >=, <>, !=,` or `LIKE`)
  - ◊ A value to compare
  - ◊ Syntax:

```
WHERE column_name [operator] value;
```

- Also permits multiple criteria to be specified with `AND` or `OR`

1.A.7 `SELECT` … `WHERE` Statements

- In a `SELECT` statement, after the `FROM` clause comes the optional **`WHERE`** clause
  - ◊ Syntax:

```
SELECT column1 [, column2, column3, etc.]
FROM table_name
WHERE column1 = 'term';
```

    - * (… or whatever comparison operator and whatever condition)

- Any column(s) in the source table can be used in the `WHERE` clause, even if they are not listed in the `SELECT` clause
  - ◊ This is because `WHERE` defines the condition for retrieving data, while `SELECT` defines the actual data to retrieve

Section 1.B

Working with Conditional Logic

1.B.1 Boolean logical operations

- Any time we include a **conditional statement** in code, we are invoking a Boolean result which determines what data matches the query
  - ◊ When you compare one thing to another, or ask about a particular quality that something has, you might frame it as a yes-or-no question, like: “Is that shirt blue?” or “Does Dana like crab cakes?”

Is that shirt blue? *Yes*

Does Dana like crab cakes? *No*

- That’s how Boolean operations work, except instead of *yes* or *no*, the operation returns a result of TRUE or FALSE
  - ◊ If the question is unanswerable, or can’t be answered with the available information, in conversation the answer might be “I dunno” – whereas in a logical operation, that might return a result of NULL
- The Boolean value TRUE equates to the integer 1, while FALSE equates to 0

1.B.2 Comparison Operators & Functions

- **Comparison operators** evaluate to TRUE or FALSE

`>`	Greater than
`>=`	Greater than or equal
`<`	Less than
`<>, !=`	Not equal
`<=`	Less than or equal
`=`	Equal to
`<=>`	NULL-safe equal to
`BETWEEN ... AND ...`	Whether a value is within a range of values
`IS`	Test a value against a Boolean
`IS NULL`	NULL value test
`LIKE`	Simple pattern matching
combined with `NOT`	e.g. `IS NOT, NOT LIKE, IS NOT NULL, NOT BETWEEN ... AND ...`

- **Comparison functions** also test conditional logic

`IN()`	Whether a value is within a set of values
`COALESCE()`	Return the first non-NULL argument
`EXISTS()`	Whether the result of a query contains any rows
`GREATEST()`	Return the largest argument
`INTERVAL()`	Return the index of the argument that is less than the first argument
`ISNULL()`	Test whether the argument is NULL
`LEAST()`	Return the smallest argument
`STRCMP()`	Compare two strings
combined with `NOT`	e.g. `NOT IN(), NOT EXISTS()`

1.B.3 Logical Operators

- **Logical operators** return a result that is TRUE or FALSE (a Boolean value), based on whether the condition described with the operator is met or not met
  - ◊ **`AND`** (or **`&&`**) – TRUE if *both* expressions are TRUE
    - `WHERE favorite_food = 'peas' AND height = 'tall'`
    - Must have peas as a favorite food *and* be tall
  - ◊ **`OR`** (or **`||`**) – TRUE if *either* expression is TRUE
    - `WHERE favorite_food = 'peas' OR height = 'tall'`
    - *Either* has peas as a favorite food *or* is tall
  - ◊ **`NOT`** (or **`!`**) – Reverses the value of any other logical operator
    - `WHERE favorite_food = 'peas' AND NOT height = 'tall'`
    - Must have peas as a favorite food and *not* be tall
  - ◊ **`XOR`** – Equivalent to `(x AND (NOT y)) or ((NOT x) and y)`
    - `WHERE favorite_food = 'peas' XOR height = 'tall'`
    - Must either have peas as a favorite food and *not* be tall – or *not* have peas as a favorite food *and* be tall

1.B.4 Using WHERE with Multiple Conditions

- Multiple conditions can be provided in a `WHERE` statement using `AND`, `OR`, or `IN`
  - ◊ Example 1:

```
SELECT title, brand FROM products
WHERE price > 40.00 AND publisher_country = 'USA';
```

  - ◊ Example 2:

```
SELECT title, brand FROM products
WHERE price > 40.00 OR publisher_country = 'USA';
```

- You can also use a combination of `AND` and `OR` operators when testing multiple conditions
  - ◊ Use parentheses to ensure that your operators are evaluated as intended
  - ◊ Example 3:

```
SELECT title, brand FROM products
WHERE price > 40.0 AND (publisher_contact = 'Karl
    Bort' OR publisher_contact = 'Ann Vang');
```

1.B.5 Using the `NOT` Operator

- The operator `NOT` can be used directly with `WHERE` to test that a single condition is *not* true
  - ◊ Example:

```
SELECT title, brand FROM products
WHERE NOT price > 40.00;
```

  - ◊ Note that logically, this is the same as:

```
SELECT title, brand FROM products
WHERE price <= 40.00;
```

- `NOT` is also used to reverse the logical test of another operator
  - ◊ e.g. `IS NOT, NOT LIKE`

1.B.6 Using the `IN` Operator

- The **`IN`** operator allows you to specify multiple values in a `WHERE` clause
  - ◊ Essentially, `IN` is a shorthand for multiple `OR` conditions
  - ◊ Example:

```
SELECT * FROM customers
WHERE cust_state IN ('WI', 'MI', 'MN');
```

  - ◊ Note that this can be quite a bit shorter than writing:

```
SELECT * FROM customers
WHERE cust_state = 'WI'
    OR cust_state = 'MI'
    OR cust_state = 'MN';
```

- You can also check for **`NOT IN`**, where multiple values are not matched
  - ◊ Example:

```
SELECT * FROM customers
WHERE cust_state NOT IN ('WI', 'MI', 'MN');
```

1.B.7 Using the **BETWEEN** Operator

- The **BETWEEN** operator selects values within a given range, inclusive of begin and end values
  - ◊ The values can be numbers, text, or dates
  - ◊ Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

- To look for values *outside* the given range, use **NOT BETWEEN**
  - ◊ Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name NOT BETWEEN value1 AND value2;
```

- `BETWEEN` can be usefully paired with other operators such as the `IN` operator, using `AND` (or `OR` or `NOT`)
  - ◊ Syntax:

```
SELECT column_name(s) FROM table_name
WHERE column_name  BETWEEN value1 AND value2
AND other_column IN (ex1, ex2, ex3);
```

1.B.8 Testing for `NULL`

- A field with a value of `NULL` has no value – which presents a logical problem when it comes to normal comparison operators
  - ◊ It is not possible to test for NULL values using operators like =, <, or >
- Instead, SQL includes the operators `IS NULL` and `IS NOT NULL`
  - ◊ **`IS NULL`** is used to test for empty values
  - ◊ **`IS NOT NULL`** is used to test for non-empty values
  - ◊ Syntax:

```
SELECT column_1, column_2
FROM table_name
WHERE column_n IS NULL;
```

or

```
SELECT column_1, column_2
FROM table_name
WHERE column_n IS NOT NULL;
```

1.B.9 Using the **LIKE** Operator

- The **LIKE** operator is used in a WHERE clause to search for a specified pattern in a given column
  - ◊ Syntax:

```
SELECT * [(column1,...] FROM table_name
WHERE column LIKE pattern;
```

- Two wildcards often used in conjunction with LIKE:
  - ◊ The percent sign (%) represents zero, one, or multiple characters
  - ◊ The underscore sign (_) represents one, single character
  - ◊ Wildcards can be used at the beginning or end of the pattern to be searched, or both
  - ◊ Example, searching for customer last names containing John or Jon, where it also might be part of a hyphenated name:

```
SELECT * FROM customers
WHERE cust_name LIKE '%john%'
      OR cust_name LIKE '%jon%';
```

1.B.10 Single Quotes & Escape Characters

- SQL uses single quotes (') to denote strings, but these can also appear as a character in the middle of names or words
  - ◊ For example, there are apostrophes, aka single quotes, in the song title *Don't Stop Believing* and in the last names O'Meara and O'Connell

- To escape a single quote in the middle of a string, use two single quotes
  - ◊ Example:

```
SELECT * FROM customers
WHERE cust_name = 'O''Meara';
```

- If the single quote is at the beginning or end of the string, use three single quotes
  - ◊ Example:

```
SELECT * FROM ff_league_roster
WHERE team_name = '''Bama Mamas';
```

1.B.11 Performing a Date Search

- Date/time values in relational databases often contain both a date and time component
  - ◊ In MySQL databases, often these are values using the `DATETIME` or `TIMESTAMP` data types
- Specific date increments can be extracted using the functions `YEAR`, `MONTH`, and `DAY`
  - ◊ Example (within a `WHERE` clause):

```
WHERE YEAR(order_date) = 2023;
```

- When performing a search using a date string that does not include time, the date string is implicitly converted to a datetime value using the time value of 00:00:00 (meaning 12:00 am)
  - ◊ This can throw off search results!
- To account for this behavior, you can search for a range of dates
  - ◊ Example:

```
SELECT *
FROM orders
WHERE order_date > '12/31/2023'
      AND order_date < '4/1/2024';
```

Section 1.C

Other Query Concepts

1.C.1 Ordering Results

- The **`ORDER BY`** keyword is used with `SELECT` to sort the results
  - ◊ At least one column name must be specified for sorting, though it does not have to be a column retrieved by the `SELECT` statement
  - ◊ If multiple columns are specified, leftmost column takes precedence
  - ◊ Syntax:

```
SELECT * [column1, column2, ...]
FROM table_name
ORDER BY column1[, column2, ...] ASC|DESC;
```

- By default, results will be sorted in ascending order (or can explicitly specify `ASC`)
  - ◊ To sort results in descending order, include keyword `DESC`
  - ◊ Example:

```
SELECT * FROM customers
WHERE cust_state IN ('WI', 'MI', 'MN')
ORDER BY cust_state DESC, cust_name ASC;
```

1.C.2 Sampling Records with **`LIMIT`**

- Often to perform a task, or to review how a query is performing, you don't want to retrieve all possible results from the database (which might contain millions of records!)

- MySQL uses the keyword **`LIMIT`** to cap the number of records retrieved with a `SELECT` statement
  - ◊ If fewer than that number of records are retrieved, only the matching records will be displayed
  - ◊ Syntax:

```
SELECT * [column1, column2, ...]
FROM table_name
[WHERE condition]
[ORDER BY column, ...]
LIMIT [number];
```

- Note that `LIMIT` does not provide *random* sampling, but instead returns the top set of results based on whatever ordering is applied, either by default or using `ORDER BY`

- Also note: ANSI SQL has not assigned a standard, so different RDBMS vendors solve this using different keywords
  - ◊ For instance, `LIMIT` is used by MySQL and PostgreSQL, while `TOP` is used by SQL Server, and other vendors may use a variation of `FETCH` or `SAMPLE`

1.C.3 Non-unique records with `DISTINCT`

- To retrieve only distinct (i.e. unique / non-duplicate) values, use **`SELECT DISTINCT`**
  - ◊ Syntax:

```
SELECT DISTINCT column
FROM table;
```

- If you'd like to find the number of distinct values, you can pair `SELECT` with the **`COUNT()`** function (a type of aggregate function):
  - ◊ Syntax:

```
SELECT COUNT(DISTINCT column)
FROM table;
```

  - ◊ Logic can start to get a little more complex, e.g., if you want to retrieve records from one column while counting nonunique records from a second column:

```
SELECT column1, COUNT(DISTINCT column2)
FROM table
GROUP BY column1;
```

    - ∗ We'll talk more about the `GROUP BY` keyword! In the above example, it's necessary to correctly display the count of column2 just once for each unique value in column1 (otherwise would result in an error)

1.C.4 Concatenation

- The **`CONCAT()`** function can be used in MySQL to concatenate records from multiple columns – that is, put them together into one string that resides in a new column
  - ◊ Only character (string) data types can be concatenated; other data types must first be cast as string, e.g. CHAR or VARCHAR
  - ◊ Because concatenation generates a new column, the column must be assigned an alias in order to have a name
  - ◊ Syntax:

```
SELECT CONCAT(column1, column2) AS column_name
FROM table;
```

- String literals also can be concatenated with column data (helpful for adding spaces or punctuation between pieces of data!)
  - ◊ Syntax including string literals:

```
SELECT CONCAT(column1, '  ', column2) AS column_name
FROM table;
```

  - ◊ Example:

```
SELECT CONCAT('Congratulations, ',
    first_name,
    ', on ',
    CAST(yrs_cust AS CHAR),
    ' years as a valued customer!')
AS 'anniversary_msg'
FROM customers
WHERE yrs_cust >= 1;
```

1.C.5 Adding Calculations to Results

- SELECT statements can be used to add **calculated fields** to the retrieved results
  - ◊ Because the calculated column does not exist in the original table, best practice is to assign it an alias for reference – otherwise the column name will use the calculation itself
  - ◊ Starting columns for a calculated field must use a numeric data type
  - ◊ Syntax:

```
SELECT column1 [+,-,*,/] column2 AS column_name
FROM table;
```

  - ◊ Example:

```
SELECT prod_id, prod_name, quantity, cost_per_unit,
       quantity * cost_per_unit AS inventory_value,
       last_ordered
FROM inventory;
```

Module 2

Querying Multiple Tables

Section 2.A

Joining Tables

2.A.1 Introducing Joins

- Relational databases typically have many related tables resulting from the normalization process, where no single table has all the information you might need

- To get information from related tables into one result set, we have to **join** them
  - ◊ A **`JOIN`** clause is used with `SELECT` to combine rows from two or more tables, based on a related column between them
  - ◊ Syntax:

```
SELECT column1, column2, column3  FROM table1
[INNER, LEFT, RIGHT, FULL] JOIN table2
ON table1.column = table2.column;
```

    - * Tables are most often joined on the primary key in one table and matching foreign key in another (but can also join on other "ad hoc" relationships not explicitly defined in the database)

- There are different types of joins in SQL:
  - ◊ (INNER) JOIN: Returns records with matching values in both tables
  - ◊ LEFT (OUTER) JOIN: Returns all records from the left table and matched records from the right table
  - ◊ RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table (less commonly used)
  - ◊ FULL (OUTER) JOIN: Returns all records when match in either table

2.A.2 Using Qualified Column Names

- Column names that exist in multiple tables (common with primary keys / foreign keys) must be **qualified** with the table name
  - ◊ Including the table name ensures that the column name is unambiguous
  - ◊ Column names that are unique in the database do not have to be prefixed with their table names; however, you might still choose to do so (e.g. for consistency or when using auto-detect tools)
- The format of a qualified column name is `tablename.column`
- If the table is given an alias in the `SELECT` statement, then the alias must be used, i.e. `tablealias.column`
  - ◊ Example:

```
SELECT cust_name, team_name
FROM customers as c
JOIN ff_league_roster as f ON c.cust_id = f.cust_id;
```

2.A.3 The `USING` Keyword

- Some RDBMS including MySQL, PostgreSQL, and Oracle permit substituting the **`USING`** clause in place of the `ON` clause

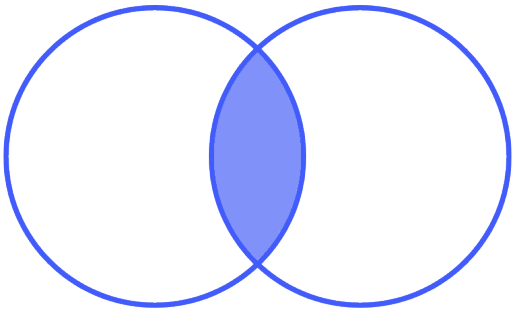
- **`USING`** can be utilized where the columns to be joined on are using an *equijoin* (that is, =), and works with inner and outer joins
  - ◊ Syntax:

```
SELECT column1, column2, ... FROM table1
JOIN table2 USING (record_id);
```

  - ◊ This format is not required, but in some situations it can help with the usability / readability of your queries

2.A.4 Inner Joins

- With an **inner join**, matching records that exist in both tables are included in the result set
  - ◊ Inner joins are the most common type of join



INNER JOIN

  - ◊ Syntax:

```
SELECT column1, column2  FROM table1
[INNER] JOIN table2
ON table1.column = table2.column;
```

    - ∗ In MySQL, the keyword “INNER” is optional, as JOIN, CROSS JOIN, and INNER JOIN are syntactic equivalents (that is, they can replace each other)

- Although a join condition usually consists of a single comparison, it’s possible to include multiple comparisons using AND or OR
  - ◊ Example:

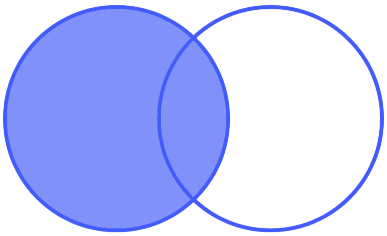
```
SELECT cust_name, team_name FROM customers as c
JOIN ff_league_roster as f ON c.cust_id = f.cust_id
       AND f.captain = TRUE;
```

2.A.5 Outer Joins

- When joining tables, records might exist in one table that don't have a match in the other table
  - ◊ For example, some inventory records might not match any customer sales, and some customers might not match any product sales
- An **outer join** retrieves all rows that satisfy the join condition *plus* unmatched rows from one or both tables
  - ◊ For an unmatched row, any columns from the other table that are included in the result set are given NULL values in the results
- The three types of outer join are `LEFT`, `RIGHT`, and `FULL`

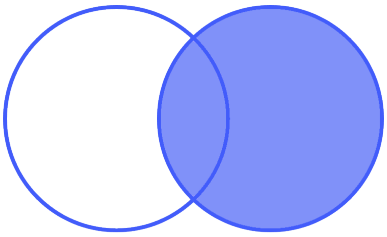
2.A.6 Types of Outer Joins

- **`LEFT JOIN`** (or left outer join): includes all records from the first table and matching records from the second table



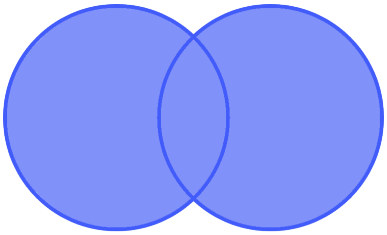
LEFT JOIN

- **`RIGHT JOIN`** (or right outer join): includes only matching records from the first table and all records from the second table



RIGHT JOIN

- **`FULL JOIN`** (or full outer join): includes all records from both tables

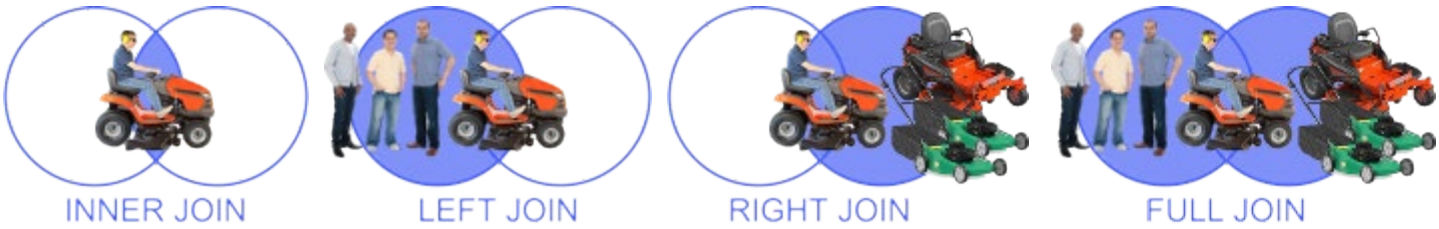


FULL JOIN

  * Note: the keyword `FULL JOIN` used by other SQL dialects but is not natively supported in MySQL
  * Instead, MySQL users can emulate a FULL JOIN with a combination of `LEFT JOIN … UNION … RIGHT JOIN`

2.A.7 Which Join to Use?

- What type of join to use depends on whether you want to include non-matching records in the result set
  - ◊ For instance, if you need to retrieve a list of customers who purchased lawnmowers, then you would want to use an **inner join**
  - ◊ If you need to return a list of *all* customers and also include information on which of them purchased lawnmowers, then you would want to use a **left outer join**
  - ◊ On the flip side, if you want to retrieve info only on customers who have purchased lawnmowers plus all info on all remaining other lawnmowers, then you'd use a **right outer join**
    - ∗ Notice it's the order of the tables that determines "left" and "right" – the first-listed table is "left" and the second-listed table is "right"
  - ◊ Lastly, if you want to return a complete list of customer info and lawnmower info, including which customers have purchased which lawnmowers, then you would use a **full outer join**
    - ∗ Again, in MySQL, use the `LEFT JOIN ... UNION ... RIGHT JOIN` workaround (we'll talk more about `UNION` later this week)



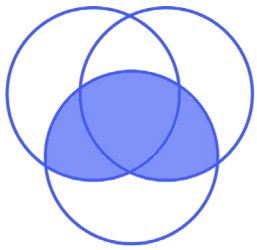
2.A.8 Multi-table Joins

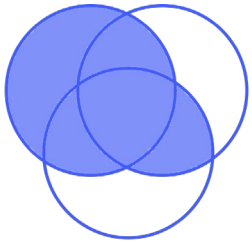
- You can join more than two tables by using multiple `JOIN` clauses
  - ◊ The first join is evaluated to a result set; those results are used as the input for the next join; and so on for any additional joins
  - ◊ Note that the order in which the joined tables appear does not determine processing; rather, the query processor evaluates the query as a whole

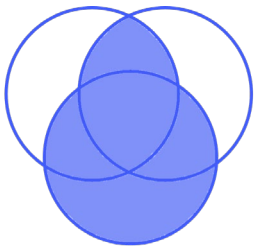
- Multiple tables may be joined to bridge relationships between distantly related tables, or to combine info from multiple tables
  - ◊ Example:

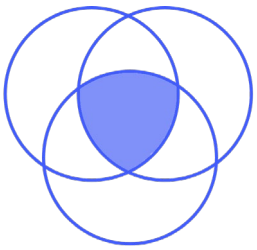
```
SELECT catalog.title FROM catalog
JOIN inventory ON inventory.book_id = catalog.book_id
JOIN store ON inventory.store_id = store.store_id
JOIN address ON store.address_id = address.address_id
WHERE city = 'Springfield';
```

- When mixing JOIN types, be mindful of the total effect!
  - ◊ Example: If TableA is LEFT JOINed to TableB, and then an INNER JOIN is used to join TableC, any row from A that had no match in B will only survive if it also has a match in C. To preserve the intended behavior, use a LEFT JOIN for the third table as well, or carefully reconsider the join order and conditions.









2.A.9 Adding `WHERE` Clauses to Joins

- When joining tables, a `WHERE` clause can be added to the query *after* all `JOIN` clauses
  - ◊ `JOIN` cannot follow `WHERE`
- `WHERE` conditions can be created for any column in any of the tables being joined
  - ◊ Syntax:

```
SELECT column1, column2  FROM table1
JOIN table2 ON table1.column = table2.column
JOIN table3 ON table3.column = table2.column
WHERE column4 = 'value';
```

2.A.10 Using Self-Joins

- A **self-join** is where a table is joined to itself

- Self-joins can be useful when working with recursive relationships (when a foreign key resides in the same table as the primary key it references)

- In order to create a self-join, the table name must be assigned a different alias for each instance to prevent ambiguity

  ◊ Syntax:

```
SELECT column1, column2  FROM table AS x
JOIN table AS y ON x.column_pk = y.column_fk;
```

Section 2.B

Creating and Using Views

2.B.1 Views

- This concept was briefly covered last week, but now you have a little more context in which to use it!

- A **view** is a stored query that is saved in the database and can be used like a table
  - ◊ Unlike a table, a view does not store data itself — it simply stores the query definition, and the underlying data is retrieved from the base tables each time the view is queried
  - ◊ Views are sometimes called "virtual tables" for this reason

- Views are useful for several purposes:
  - ◊ Simplifying complex queries — a frequently used multi-table join or complex SELECT can be saved as a view, allowing users to query it like a simple table
  - ◊ Restricting access to data — a view can expose only certain columns or rows from a table, allowing administrators to control what data different users can see without changing the underlying table structure
  - ◊ Presenting data in a more readable or useful format — for instance, a view might combine first and last name columns into a single FullName field, or apply a CASE structure to translate coded values into readable labels

- Note that views can also be based on other views, not just tables
  - ◊ This is occasionally useful, but beware that deeply nested views can create performance and maintenance challenges

2.B.2 Creating a View

- In MySQL, a view is created using the command **`CREATE VIEW`**
  - ◊ Syntax:
    ```
    CREATE VIEW view_name AS
    SELECT column1, column2, ...
    FROM table_name
    WHERE condition;
    ```
  - ◊ Example:
    ```
    CREATE VIEW ActiveCustomers AS
    SELECT CustomerID, CustomerName, Email
    FROM Customers
    WHERE Active = TRUE;
    ```
- Once created, the view can be queried just like a table:
  - ◊ Example:
    ```
    SELECT * FROM ActiveCustomers;
    ```

2.B.3 Modifying or Deleting a View

- Views can be modified using **`CREATE OR REPLACE VIEW`**, which replaces an existing view definition without needing to drop it first
  - ◊ This command is a convenient shorthand that combines two operations into one: if the view already exists, it replaces the existing definition, and if the view doesn't exist yet, it creates it fresh
  - ◊ This can be used to alter the view query itself, but cannot change the names or data types of the columns that the view exposes (instead, you would need to `DROP` and recreate the view)
- A view can be removed using **`DROP VIEW`**
- Important limitations to be aware of:
  - ◊ Most views in MySQL cannot be used to directly update records — that is, you generally cannot use INSERT, UPDATE, or DELETE on a view
    - * For this reason, views are primarily used for *reading* data rather than modifying it
  - ◊ Views do not improve query performance on their own, since the underlying query still runs each time the view is accessed
- For detailed documentation on MySQL views, see: https://dev.mysql.com/doc/refman/8.0/en/views.html

Module 3

Writing Complex Queries

Section 3.A

Using Aggregate Functions

3.A.1 Aggregate Functions

- Aggregate functions operate on multiple values to return a single answer, such as a total of values in a column, or a count of records from one column that match a record in another column
  - ◊ Aggregate functions are often used with the `GROUP BY` clause to split the result-set into groups of values, where the aggregate function can be used to return a single value for each group

- Common functions for aggregation include:
  - ◊ **`MIN()`** - returns the smallest value within the selected column, or for string data, the nearest to A alphabetically
  - ◊ **`MAX()`** - returns the largest value within the selected column, or for string data, the nearest to Z alphabetically
  - ◊ **`SUM()`** - adds up the total sum of a numerical column
  - ◊ **`AVG()`** - returns the average value of a numerical column
  - ◊ **`COUNT()`** - returns the number of rows in a set

- Columns that return results of aggregation should typically be assigned an alias for clarity
  - ◊ Examples of syntax:

```
SELECT SUM(column) AS 'Sum of X'
FROM table;

SELECT column1, COUNT(DISTINCT column2) AS 'Count of X'
FROM table
GROUP BY column1;
```

3.A.2 Tips when using aggregate functions

- For all the aggregate functions, NULL values are ignored by default
  - ◊ In order to include these records in an aggregate function, values first would need to be converted to zero

- For `AVG` and `SUM`, the `DISTINCT` keyword can be helpful to include only the unique occurrences of a value

- For `COUNT`, there are several variations that can be useful:
  - ◊ `COUNT(*)` returns the total number of records in a table
  - ◊ `COUNT(column_name)` or `COUNT(ALL column_name)` returns the total number of non-null values in a column, including duplicates
    - * The keyword `ALL` is default for `COUNT` and so is optional
  - ◊ `COUNT(DISTINCT column_name)` returns the total number of distinct or unique values in a column

3.A.3 Grouping Results

- Results of aggregate functions can be grouped by other columns in the table to display useful subtotals

- When using **`GROUP BY`**:
  - ◊ Must include the name of the column(s) used for grouping results
    - ∗ Note that the column being grouped on does not have to be included in the `SELECT` clause (although it usually is)
  - ◊ All columns appearing in the SELECT clause that are not part of an aggregate function must be included in the `GROUP BY` clause
    - ∗ In grouping, left-most column takes precedence
  - ◊ Typically columns created by aggregate function should be assigned an alias to give them a more reader-friendly name
  - ◊ If a WHERE clause is being included in the query, GROUP BY should *follow* WHERE
  - ◊ If an ORDER BY clause is being included in the query, GROUP BY should *precede* ORDER BY
  - ◊ Syntax (using `[FX]` to stand in for an aggregate function):

```
SELECT column1, [FX](DISTINCT column2) AS '[FX] of X',
       column4, column5 [...]
FROM table
WHERE column = value
GROUP BY column1, column5, column4 [...]
ORDER BY column4;
```

3.A.4 **HAVING** Filter for Aggregated Columns

- To add filtering to a column being aggregated, a **HAVING** clause can be used
  - ◊ HAVING is like WHERE for aggregated columns
    - * WHERE can only be used to filter columns that exist in the database, and cannot be used to filter aggregated columns
    - * HAVING can also filter on other columns; however, best practice is to use WHERE when possible and HAVING only as needed on aggregates
  - ◊ In a query, HAVING should *follow* the GROUP BY clause
    - * Syntax:

```
SELECT column1, [FX](DISTINCT column2) AS FX_of_X
FROM table
WHERE column [OPERATOR] value
GROUP BY column1
HAVING FX_of_X [OPERATOR] value
ORDER BY columnZ;
```

Section 3.B

Scalar Functions

3.B.1 Overview of Scalar Functions

- Scalar functions allow you to perform different calculations on data values per record/row
  - ◊ These functions operate on rows individually and produce one result (scalar) per row
- Categories of scalar functions:
  - ◊ String functions perform operations on character values
  - ◊ Numeric functions perform operations on numeric values
  - ◊ Date functions perform operations on date/time values
  - ◊ Conversion functions convert column data types
  - ◊ NULL-related functions provide options for handling null values

3.B.2 Data Type Conversion Functions

- Conversion functions convert column data types
  - ◊ See reference list: https://www.w3schools.com/sql/sql_ref_mysql.asp (scroll to bottom for conversion functions under "Advanced Functions")
- Types of conversion function:
  - ◊ **`CAST`** converts a value (of any type) into a specified datatype
  - ◊ **`CONV`** converts a number from one numeric base system to another
  - ◊ **`CONVERT`** converts a value into the specified datatype or character set

3.B.3 String Functions

- String functions perform operations on character values
  - ◊ A handy reference list of MySQL string functions can be found at: https://www.w3schools.com/sql/sql_ref_mysql.asp

- Examples of string functions include:
  - ◊ CONCAT_WS adds two or more expressions together with a separator
  - ◊ FORMAT formats a number to a format like "#,###,###.##", rounded to a specified number of decimal places
  - ◊ INSERT inserts a string within a string at the specified position and for a certain number of characters
  - ◊ LCASE or LOWER converts a string to lower-case
  - ◊ REPLACE replaces all occurrences of a substring within a string, with a new substring
  - ◊ REVERSE reverses a string and returns the result
  - ◊ STRCMP compares two strings
  - ◊ SUBSTR or SUBSTRING or MID extracts a substring from a string (starting at any position)
  - ◊ TRIM removes leading and trailing spaces from a string (alt RTRIM, LTRIM)
  - ◊ UCASE or UPPER converts a string to upper-case

3.B.4 Date Functions

- Date functions perform operations on date/time values
  - ◊ A handy reference list of MySQL date/time functions can be found at: https://www.w3schools.com/sql/sql_ref_mysql.asp

- Examples of date functions include:
  - ◊ CURDATE or CURRENT_DATE returns the current date
  - ◊ CURTIME or CURRENT_TIME returns the current time
  - ◊ CURRENT_TIMESTAMP returns the current date and time
  - ◊ DATE extracts the date part from a datetime expression
  - ◊ DATEDIFF returns the number of days between two date values
  - ◊ DATE_ADD or ADDDATE adds a time/date interval to a date and then returns the date
  - ◊ DATE_FORMAT Formats a date
  - ◊ DATE_SUB or SUBDATE subtracts a time/date interval from a date and then returns the date
  - ◊ DAYNAME returns the weekday name for a given date
  - ◊ DAYOFWEEK returns the weekday index for a given date
  - ◊ DAYOFYEAR returns the day of the year for a given date
  - ◊ QUARTER returns the quarter of the year for a given date value

3.B.5 NULL-related functions

- NULL-related functions provide options for handling null values
  - ◊ See reference list: https://www.w3schools.com/sql/sql_ref_mysql.asp (scroll to bottom for null-related functions under "Advanced Functions")
- Examples:
  - ◊ COALESCE returns the first non-null value in a list
  - ◊ IFNULL returns a specified value if the expression is NULL, otherwise returns the expression
  - ◊ ISNULL returns 1 or 0 depending on whether an expression is NULL
  - ◊ NULLIF compares two expressions and returns NULL if they are equal; otherwise, the first expression is returned

3.B.6 Numeric/Math Functions

- Numeric functions perform operations on numeric values
  - ◊ A handy reference list of MySQL numeric functions can be found at: https://www.w3schools.com/sql/sql_ref_mysql.asp

- Examples of numeric functions:
  - ◊ ABS returns the absolute value of a number
  - ◊ GREATEST returns the greatest value of the list of arguments
  - ◊ LEAST returns the smallest value of the list of arguments
  - ◊ MOD returns the remainder of a number divided by another number
  - ◊ PI returns the value of PI
  - ◊ POW or POWER returns the value of a number raised to the power of another number
  - ◊ RAND returns a random number
  - ◊ SQRT returns the square root of a number
  - ◊ Rounding functions like CEILING, FLOOR, ROUND, TRUNCATE

Section 3.C

Set Operations

3.C.1 What Are Set Operations?

- **Set operations** allow you to combine the results of two or more `SELECT` statements into a single result
  - ◊ The result of each `SELECT` statement is a set – combining multiple sets requires a set operation
  - ◊ This is different from joins, where each join combines columns from multiple tables into a single result set
- In order to be combined with a set operator, each `SELECT` statement must return the same number of columns, and the data types of the columns must be compatible
- Set operators include `UNION`, `INTERSECT` and `EXCEPT`
  - ◊ Note: `INTERSECT` and `EXCEPT` were added to MySQL in 2022 with version 8.0.31, so older versions of MySQL do not support these keywords

3.C.2 `UNION` Operator

- **`UNION`** combines the result sets of two `SELECT` statements into a single result set by appending the rows of one beneath the other, omitting any duplicate rows
  - ◊ To retain duplicates, use **`UNION ALL`** instead.
  - ◊ The column names returned with the result set are from the first set
- Useful where data is spread across multiple tables (such as separate sales tables for each month) or where multiple tables contain similar data
- The number of columns, order of columns, and data types of each column *must be the same* for all `SELECT` statements whose results are being unioned
  - ◊ If using `ORDER BY`, follows the last given `SELECT` statement
  - ◊ Syntax:
    ```
    SELECT column_name(s) FROM table_1
    UNION
    SELECT column_name(s) FROM table_2;
    ```
  - ◊ Example:
    ```
    SELECT 'customer' AS name, contact, city, country
    FROM customers
    UNION
    SELECT 'supplier', contact, city, country
    FROM vendors
    ORDER BY country, city;
    ```

3.C.3 Using `INTERSECT`

- **`INTERSECT`** combines only those rows which the results of two query blocks have in common, omitting any duplicates (effectively like an inner join)
  - ◊ Same general rules apply as when using `UNION` operator: the number of columns, order of columns, and data types of each column must be the same for all `SELECT` statements
  - ◊ Syntax:

```
SELECT column_name(s) FROM table_1
INTERSECT
SELECT column_name(s) FROM table_2;
```

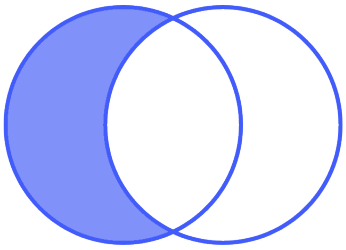
  - ◊ Example:

```
SELECT 'customer' AS type, contact, city, country
FROM customers
INTERSECT
SELECT 'supplier', contact, city, country
FROM vendors
ORDER BY contact;
```

- **`INTERSECT ALL`** returns all matching values *including* duplicates

3.C.4 Using **EXCEPT**

- To filter to results that belong only to the first `SELECT` statement but not the second, you can use the **`EXCEPT`** operator
  - ◊ That is, for `SELECT` statements A and B, `EXCEPT` returns all results from A which are not also present in B, omitting any duplicates



EXCEPT

  - ◊ Note: a few RDBMS vendors like Oracle use `MINUS` instead of `EXCEPT`
  - ◊ Syntax:

```
SELECT column_name(s) FROM table_1
EXCEPT
SELECT column_name(s) FROM table_2;
```

  - ◊ Example:

```
SELECT customer AS org, contact, city, country
FROM customers
EXCEPT
SELECT vendor AS org, contact, city, country
FROM vendors
ORDER BY contact;
```

- **`EXCEPT ALL`** returns all matching values *including* duplicates

Module 4

Using Subqueries

Section 4.A

Working with Subqueries

4.A.1 Overview of Subqueries

- Queries can get very complex! To manage this complexity, **subqueries** can be used by nesting one query within another
  - ◊ A subquery is a `SELECT` statement embedded within another query
  - ◊ The nested query is referred to as the **inner query**, while the statement containing the subquery is referred to as the **outer query**
    - * Inner and outer are relative terms in statements with multiple nested subqueries
  - ◊ Many subqueries can be run independently of the outer query

- A subquery must always be enclosed within parentheses
  - ◊ Syntax (example):

```
SELECT column1, column2  FROM table_1
WHERE column1 IN
    (SELECT column1 FROM table_1
     WHERE column3 = 'value');
```

- In general, a subquery can be used anywhere in the query where an expression is allowed, such as within a `SELECT` clause, `FROM` clause, `HAVING` or `WHERE` clause, or nested in another subquery
  - ◊ Subqueries can also be used with an `INSERT`, `UPDATE`, or `DELETE` statement
  - ◊ For MySQL restrictions on subqueries, see documentation: https://dev.mysql.com/doc/refman/8.0/en/subquery-restrictions.html

4.A.2 Subqueries as Column Expressions

- A subquery can be used in the initial clause (typically the column list) of a SELECT statement

- Used like this, the subquery must return a single value (i.e. one row, one column)

- Typically an aggregate function or WHERE clause is used to guarantee the subquery returns only one value
  - ◊ Example:

```
SELECT prod_name, price, (
    SELECT AVG(price) FROM products
    ) AS average_price
FROM products;
```

4.A.3 Subqueries in `FROM` or `JOIN` Clauses

- If a subquery returns multiple rows and/or columns, it can be used wherever a table is expected – this includes within a FROM clause or joined with a JOIN statement
  - ◊ A subquery used in this way is often referred to as a **derived table**
  - ◊ This may be a helpful technique to speed up query execution for joins when the size of the derived table can be significantly reduced from the original full table
  - ◊ Syntax:

```
SELECT column1, column2  FROM table1
JOIN (SELECT column3, [FX](column4) AS func_of_B
      FROM table 2
      GROUP BY column3) AS subquery1
ON table1.column1 = subquery1.column3;
```

4.A.4 Nesting Subqueries

- A subquery can also include one or more **nested subqueries**
  - ◊ Be cautious about overdoing it with nesting, as subqueries that are nested more than two or three levels deep can become difficult to read!
  - ◊ Different RDBMS vendors may apply restrictions on nesting subqueries (for example, SQL Server limits nesting to 32 levels deep) – MySQL does not expressly provide any limitations on nesting
  - ◊ Syntax:

```
SELECT column1, column2  FROM table1
JOIN (SELECT column3, [FX](column4) AS func_of_B
      FROM table_2
      WHERE column3 IN
          (SELECT column5 FROM table_2
           WHERE column5 = 'value')
      ) AS subquery1
ON table1.column1 = subquery1.column3;
```

4.A.5 Subqueries in Other Statements

- Subqueries are not limited only to use in `SELECT` statements; they can also be used within `INSERT`, `UPDATE`, or `DELETE` statements

- With `INSERT`: a subquery can be used to populate a table with results from another table

  ◊ Example: inserting a set of high-value customers into a separate priority customer table:

  ```
  INSERT INTO PriorityCustomers (CustID, CustName, Email)
  SELECT CustID, CustName, Email
  FROM Customers
  WHERE CustID IN (
      SELECT CustID FROM Orders
      GROUP BY CustID
      HAVING SUM(OrderTotal) > 10000
  );
  ```

  Here the subquery identifies customers whose total order value exceeds $10,000, and the `INSERT` populates the PriorityCustomers table with their information

4.A.6 Subqueries in Other Statements (cont.)

- With `UPDATE`: a subquery can be used to determine which records to update, or to supply the new value
  - ◊ Example: marking products as discontinued if they have had no orders in the past two years:

```
UPDATE Products
SET Discontinued = TRUE
WHERE ProductID NOT IN (
    SELECT DISTINCT ProductID FROM OrderDetails
    WHERE OrderDate >= '2023-01-01'
);
```

    Here the subquery returns all ProductIDs that have appeared in a recent order, and the UPDATE applies to any product not in that list

- With `DELETE`: a subquery can be used to identify which records to delete based on conditions in another table
  - ◊ Example: removing cancelled orders from the order details table:

```
DELETE FROM OrderDetails
WHERE OrderID IN (
    SELECT OrderID FROM Orders
    WHERE Status = 'Cancelled'
);
```

    Here the subquery identifies all cancelled OrderIDs from the Orders table, and the `DELETE` removes the corresponding line items from OrderDetails

  - ◊ Note: as always with `DELETE`, it is good practice to first run the subquery as a standalone `SELECT` to confirm the records being targeted before executing the delete

4.A.7 Types of Subqueries

- There are two types of subqueries: simple and correlated
- A **simple subquery** can be evaluated independently of the outer query and is processed once for the entire statement
- A **correlated subquery** cannot be evaluated independently of the outer query because it depends on data from the outer query to run
  - ◊ This means a correlated subquery runs for *each row* of the outer query
- For example, say you want to find all products that have ever been ordered using a Products table and an OrderDetails table:
  - ◊ **Simple subquery**: the inner query runs once and produces a fixed list of ProductIDs, which the outer query then uses:

```
SELECT ProductName FROM Products
WHERE ProductID IN (
    SELECT DISTINCT ProductID FROM OrderDetails
);
```

  - ◊ **Correlated subquery**: the inner query references the outer query's current row, and runs once for each product being evaluated:

```
SELECT ProductName FROM Products p
WHERE EXISTS (
    SELECT 1 FROM OrderDetails od
    WHERE od.ProductID = p.ProductID
);
```

  - ◊ This is why correlated subqueries can be slower on large datasets; we'll talk more about query efficiency concepts covered later in this module

4.A.8 Comparing Subqueries and Joins

- Many subqueries can be rewritten as JOIN statements, and many JOIN statements can be rewritten as subqueries
  - ◊ In general, a JOIN is most often considered more efficient
  - ◊ But if you can write a query using either method, you may want to test both to see if there is a difference in performance
- Advantages of joins:
  - ◊ Columns can be returned in the result set from multiple tables (whereas columns that only appear in a subquery cannot be returned)
  - ◊ Use of `JOIN` tends to be more intuitive when using an existing relationship between tables
  - ◊ A query using `JOIN` typically performs faster than the equivalent query using a subquery
- Advantages of subqueries:
  - ◊ Can be used to pass an aggregate value to the outer query
  - ◊ May be more intuitive when using an ad hoc relationship between tables
  - ◊ Can help to break down long, complex queries into compartmented steps

4.A.9 Testing Set Membership with `IN`

- The IN operator can be used to compare the results of a subquery
  - ◊ Compared values must have the same data type (or be implicitly convertible to the same data type)
  - ◊ Syntax:

```
SELECT column1 [, ...]
FROM table_name
WHERE [HAVING] expression [NOT] IN (subquery);
```

    - * If the value of expression equals any value returned by the subquery, the `IN` condition (or `NOT IN` condition) evaluates to `TRUE`
    - * If there are no matches, or the subquery returns only null values, or the subquery result is empty, then the IN condition evaluates to `FALSE`

4.A.10 Testing Existence with `EXISTS`

- You can also test for the existence of rows in a subquery result
  - ◊ An existence test does not compare values so no test expression is required
  - ◊ The convention when testing for existence is to use `SELECT *` because you are not concerned with specific values
- When using `EXISTS` to look for rows:
  - ◊ If the subquery returns at least one row, `EXISTS` evaluates to TRUE (this includes a subquery row containing only NULL values)
  - ◊ If the subquery returns no rows, `EXISTS` evaluates to FALSE
- Similarly, you can test for nonexistence using `NOT EXISTS`

Section 4.B

Common Table Expressions

4.B.1 Common Table Expressions (CTEs)

- A Common Table Expression or **CTE** is a temporary named result set that exists only for the duration of a single query
  - ◊ CTEs are defined using the `WITH` keyword, followed by a name and a query definition, before the main `SELECT` statement
  - ◊ Like a view, a CTE can be queried like a table — but unlike a view, it is not saved to the database and exists only for the current query
  - ◊ Like a subquery, a CTE is temporary — but unlike a subquery, it is defined at the top of the query rather than nested within it, which often makes complex queries easier to read and maintain
  - ◊ Syntax:

```
WITH cte_name AS (
      SELECT column1, column2
      FROM table_name
      WHERE condition
  )
  SELECT * FROM cte_name;
```

- Multiple CTEs can be defined in the same query, separated by commas:

```
WITH cte1 AS (
      SELECT ...
  ),
  cte2 AS (
      SELECT ... FROM cte1 ...
  )
  SELECT * FROM cte2;
```

4.B.2 More on CTEs

- As you continue writing queries in a script, later CTEs can reference earlier ones, which allows you to build up complex logic in readable steps

- CTEs are particularly useful for:
  - ◊ Breaking down complex queries into logical, named steps that are easier to read and debug
  - ◊ Replacing deeply nested subqueries with something more maintainable
  - ◊ Situations where the same subquery would otherwise need to be repeated multiple times in a query

- Like views, CTEs do not inherently improve query performance — the underlying query still runs when the CTE is referenced
  - ◊ However, cleaner code structure often makes it easier to identify and fix performance issues

- For detailed documentation on MySQL CTEs, see: https://dev.mysql.com/doc/refman/8.0/en/with.html

Section 4.C

More Query Options

4.C.1 CREATE TABLE … AS SELECT

- To copy data from one table into a new table, MySQL pairs the `CREATE TABLE` keyword with `AS SELECT`
  - ◊ Syntax:

```
CREATE TABLE newtable
AS SELECT * [or column1, column2, …]
FROM oldtable
[optional: WHERE condition];
```

- `AS SELECT` can be used:
  - ◊ To create a backup copy of a table
  - ◊ To copy a table into a different database
  - ◊ To create a smaller table with only a subset of the original table's columns or records
  - ◊ To combine selected data from more than one table into a new table
  - ◊ To create a new table with no records that copies the structure of the original table (by including `WHERE` clause that returns no data)

4.C.2 INSERT INTO … SELECT

- We just talked about `AS SELECT`, which copies data from a source table into a *new* table

- **`INSERT INTO … SELECT`** copies data from a source table and inserts it into an *existing* target table
  - ◊ Data types in original and target tables must match
  - ◊ Existing records in the target table are not affected
  - ◊ Syntax:

```
INSERT INTO target_table [(target_col1, target_col2, …)]
SELECT source_column1 [, source_column2, …]
FROM source_table;
```

4.C.3 TRUNCATE TABLE

- To quickly empty a table of records, the `TRUNCATE TABLE` statement can be used
  - ◊ Syntax:
    ```
    TRUNCATE TABLE table_name;
    ```
- Logically, `TRUNCATE TABLE` is similar to a `DELETE` statement that deletes all rows, or a set of `DROP TABLE` and `CREATE TABLE` statements
  - ◊ Although `TRUNCATE TABLE` is similar to `DELETE`, it is classified as a DDL statement rather than a DML statement
- Because truncate operations drop and re-create the table, they can be much faster than deleting rows one by one, particularly for large tables
  - ◊ However, `TRUNCATE` *cannot* be performed on tables with parent-child foreign key relationships
  - ◊ `TRUNCATE` *can* be run on a table that includes only foreign keys creating recursive relationships

Section 4.D

Testing for Efficiency

4.D.1 Testing for Efficiency

- The first goal in writing SQL (or any code!) is to produce the correct desired results … but ideally we also want to do so as efficiently as possible

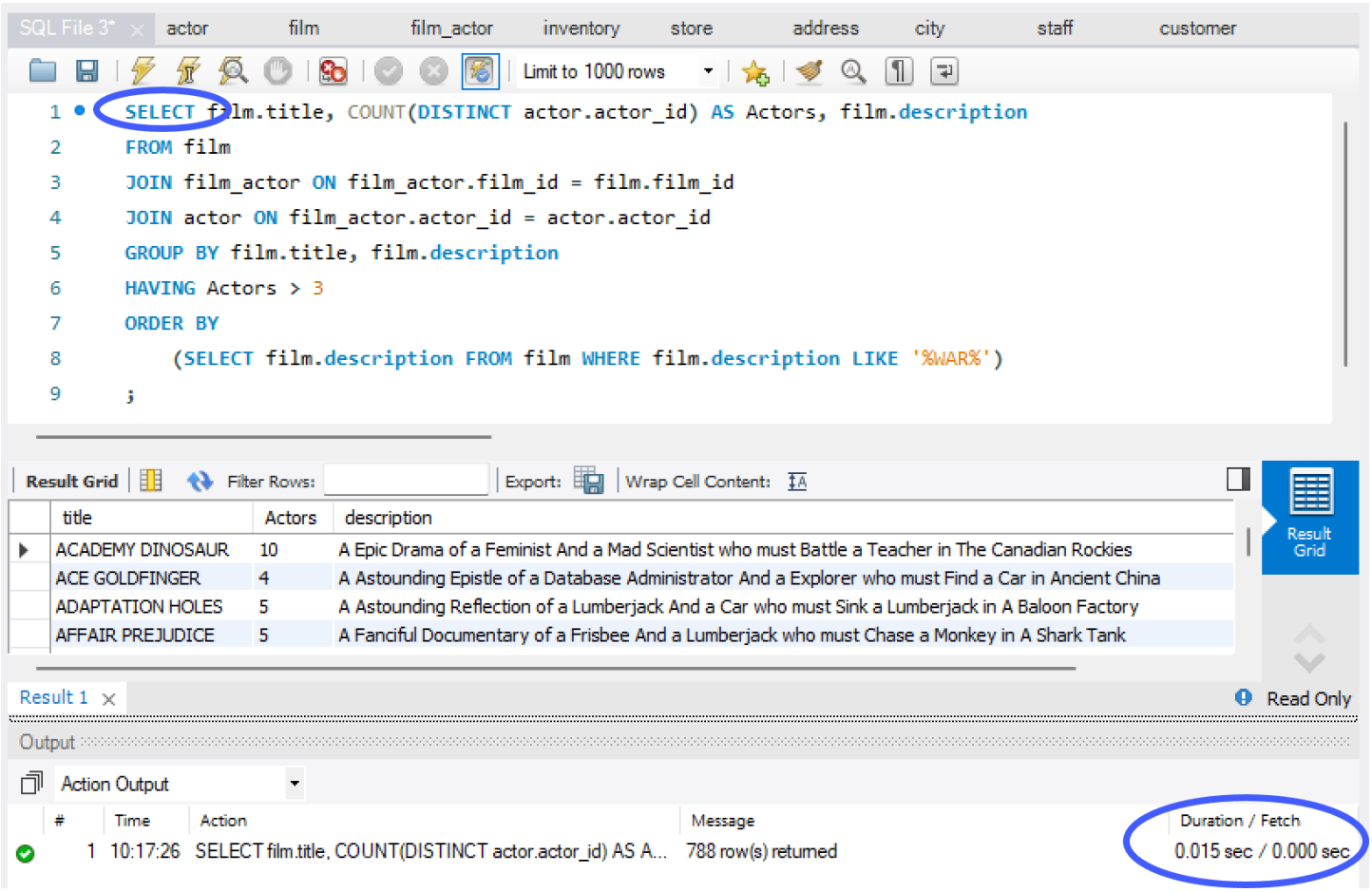
- Why does efficiency matter?
  - ◊ As we've discussed, databases can be VERY large
  - ◊ Differences in query efficiency may not be noticeable when dealing with only a few thousand records – but when you have millions or billions of records to search, efficiency becomes much more significant!

- If there is more than one way of writing a query that will produce the results we want, how do we decide which way is most optimal?
  - ◊ We will not go deep into query tuning, which is beyond the scope of this course, but these are good concepts to be aware of!
  - ◊ Note that while there are some broadly applicable principles, the most effective techniques for tuning SQL queries are platform-specific

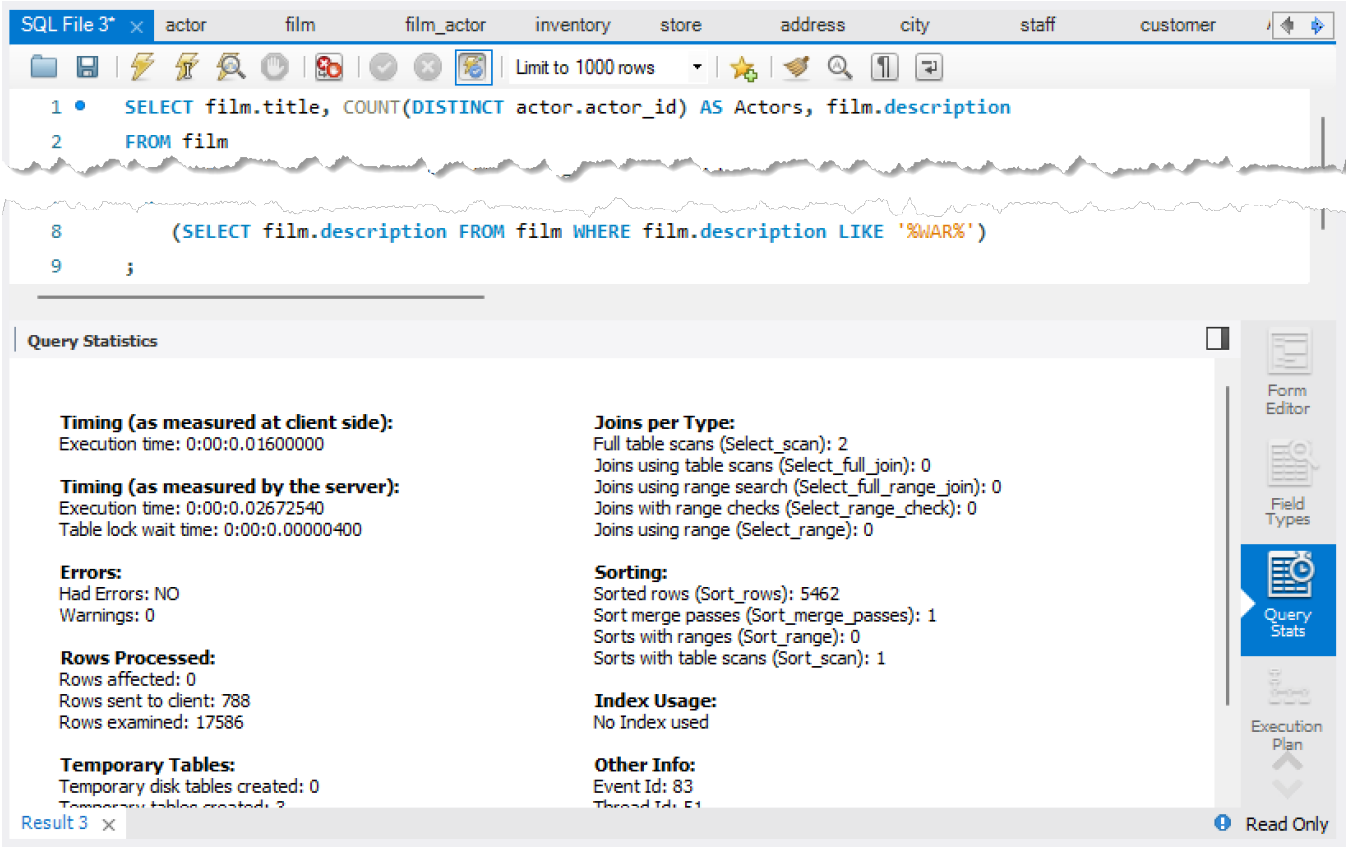
4.D.2 Performance Tools in MySQL Workbench

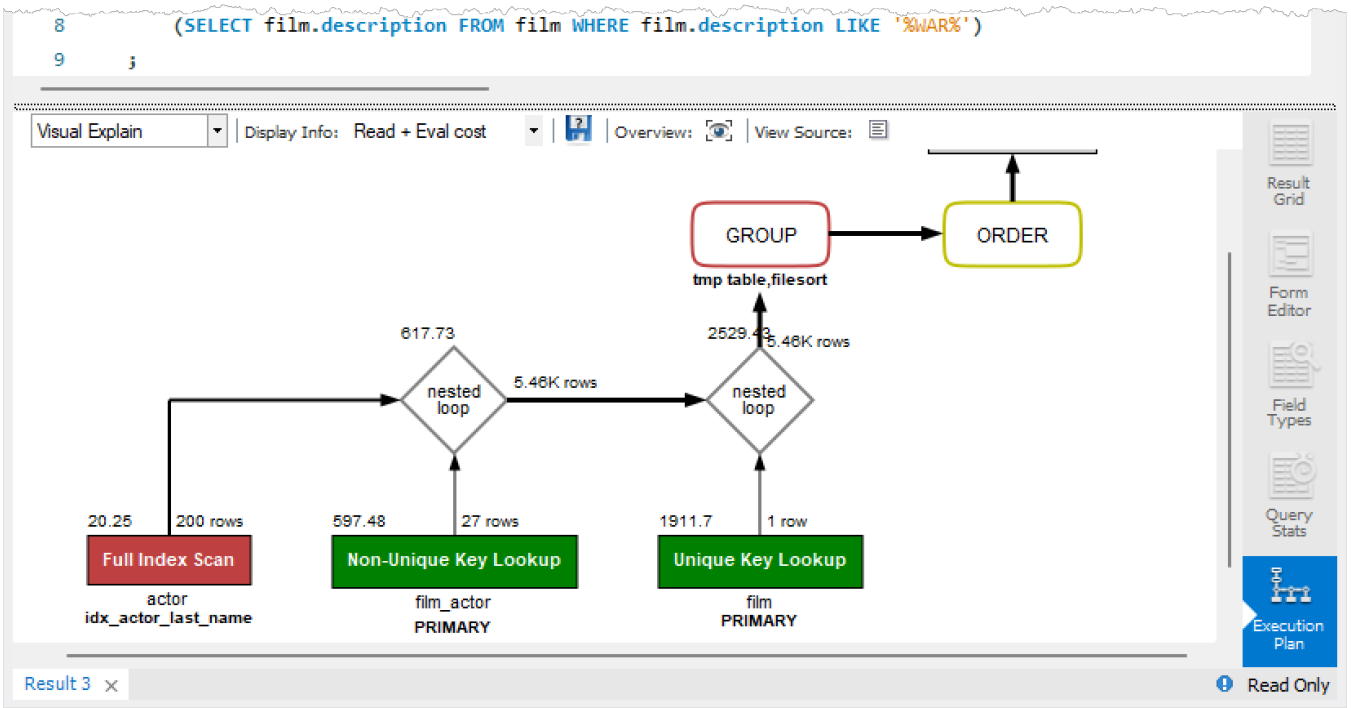
- You can access database-level performance reports from the Administration tab of the Navigator panel, under Performance

- For a specific query, the Output window displays basic information about execution speed in the Duration / Fetch column



- You learn more about query performance by going to the Query menu and choosing **Explain Current Statement:**

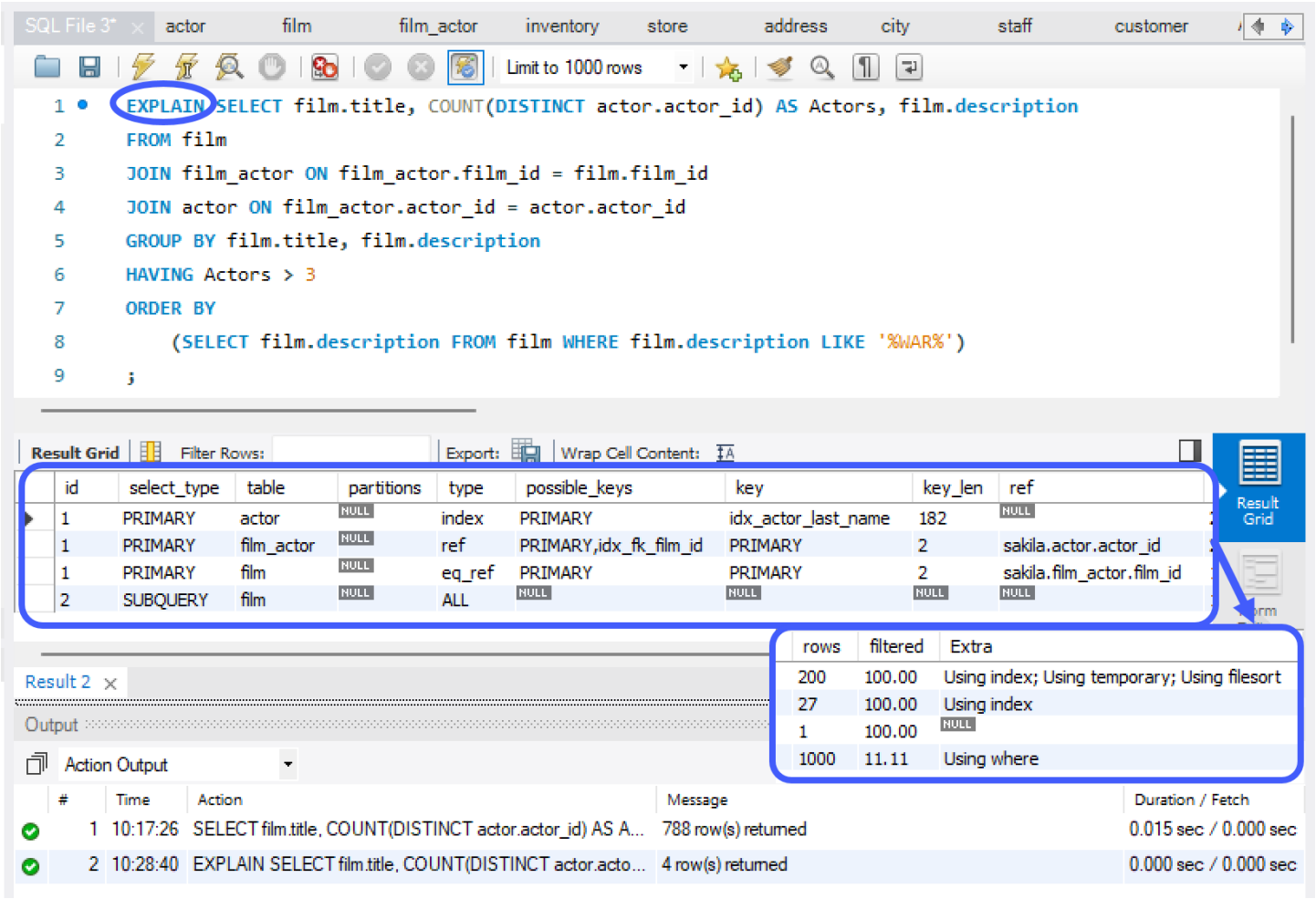




* See documentation: https://dev.mysql.com/doc/workbench/en/wb-tutorial-visual-explain-dbt3.html

4.D.3 Querying Query Efficiency

- To show the execution plan for a query in MySQL, we can use the command **`EXPLAIN`**



◊ Other RDBMS may use other keywords – for example, while PostgreSQL also uses `EXPLAIN`, Oracle uses `EXPLAIN PLAN`, and SQL Server uses `SET SHOWPLAN_TEXT ON`

Section 4.E

Working with Programming Logic

4.E.1 Programming Logic Concepts

- While the primary purpose of SQL is to manipulate and retrieve data, it also has other logic elements common to many programming languages

- These include:
  - ◊ Variables
  - ◊ Control flow statements
  - ◊ User-defined functions

- Programming logic may be implemented:
  - ◊ in a **stored procedure** which is saved to the database and which can be executed by calling its name
  - ◊ in a saved SQL script which can be imported and run
  - ◊ in a current SQL script being created for an immediate task

4.E.2 Variables

- **Variables** are used to store data within a script or procedure
  - ◊ Essentially, a variable is a memory storage container for a piece of information the program will need to use later
- User variables may be used in most contexts where expressions are permitted
  - ◊ In MySQL, user variables cannot be used directly in an SQL statement as an identifier or as part of an identifier, such as in contexts where a table or database name is expected
  - ◊ See https://dev.mysql.com/doc/refman/8.0/en/user-variables.html for more details and exceptions
- A variable that holds a single value is known as a **scalar variable**
- The **scope** of a variable is session-specific and limited to the batch of SQL statements that are being sent at one time to the server for execution

4.E.3 Declaring and Assigning Variables

- Syntax for declaring and assigning variables differs among RDBMS

- In MySQL, user variables are written starting with the @ symbol: **`@name`**
  - ◊ User variables can be assigned a value from a limited set of data types: integer, decimal, floating-point, binary or nonbinary string, or NULL value
  - ◊ The variable name must consist of alphanumeric characters, `.`, _, and `$`
    - * A variable name can also contain other characters if placed in quotes as a string or identifier (`@'my-var'`, `@"my-var"`, or `@`my-var`)
  - ◊ Examples:

```
SET @var1 = 2;

SET @var2 = @var1 * 2;
```

- To output the value of a variable, use: `SELECT @name AS '';`

- See MySQL documentation on creating user-defined variables at: https://dev.mysql.com/doc/refman/8.0/en/user-variables.html
  - ◊ Note that previous releases of MySQL let users to assign a value to a user variable in statements other than `SET`, but this functionality has been deprecated and will not be supported in future releases

4.E.4 Control flow tools

- **Control flow** refers to the way a program manages the order in which statements are executed

- Programming constructs for control flow
  - ◊ Conditional statements
    - * `CASE`
    - * `IF`
  - ◊ Looping mechanisms
    - * `LOOP`
    - * `WHILE`
    - * `REPEAT`
    - * `ITERATE`
    - * `LEAVE`

- Conditional logic and loops allow for complex decision-making and iterative processing within SQL scripts and stored procedures

- MySQL supports the `IF`, `CASE`, `ITERATE`, `LEAVE` `LOOP`, `WHILE`, and `REPEAT` constructs for flow control within stored programs and `RETURN` within stored functions
  - ◊ See reference: https://dev.mysql.com/doc/refman/8.4/en/flow-control-statements.html

4.E.5 CASE Structures

- **CASE** allows you to customize the output of a column based on one or more true-false conditions
  - ◊ The CASE statement goes through the given conditions and returns a value when the first condition is met; once a condition is true, it will stop reading and return the result
  - ◊ If no conditions are true, it will return the value of the ELSE clause (or if no ELSE clause provided, then NULL)
  - ◊ The CASE statement must conclude with the keyword END

- Using CASE structures:
  - ◊ Syntax:

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    [WHEN condition3 THEN result 3 ...]
    ELSE result
END;
```

  - ◊ Example:

```
SELECT YEAR(date) as 'Year', CASE MONTH(date)
    WHEN 1 THEN 'Jan'
    WHEN 2 THEN 'Feb'
    WHEN 3 THEN 'Mar'
    WHEN 4 THEN 'Apr'
    [... through WHEN 12 THEN 'DEC']
    END AS 'Month'
FROM date_table;
```

4.E.6 Searched CASE

- A searched CASE structure evaluates a set of logical expressions
  - ◊ Syntax:

```
CASE
    WHEN expression THEN 'result to return'
    WHEN expression THEN 'result to return'
    ELSE 'result to return if no matches'
    END;
```

  - ◊ Example:

```
SELECT product_id, quantity,
CASE
    WHEN quantity < 1 THEN 'out of stock'
    WHEN quantity < 10 THEN 'low stock'
    ELSE 'more than 10 in stock'
    END
FROM inventory;
```

4.E.7 Overview of Conditional Execution

- Sometimes in managing database content, it is helpful to execute SQL statements based on a particular condition occurring (or not occurring)
  - ◊ For instance, an inventory update might need to incorporate logic to return that an item is available if it has a quantity of greater than 5; “low stock” if quantity 1-5; and unavailable / “out of stock" if the quantity is zero
- Another important form of conditional logic is the **`IF...ELSE`** block
  - ◊ Most RDBMS support some form of `IF...ELSE` construct
- In an `IF...ELSE` block:
  - ◊ The `IF` keyword is followed by a condition that evaluates to true or false (known as a Boolean expression), and value to return if true
    - * If the `IF` condition evaluates to true, the next part of the `IF` statement is executed
    - * If the `IF` condition evaluates to false, then the next `ELSEIF` condition is checked (if there is one) or final `ELSE` statement is executed (if there is one)

4.E.8 `IF`…`ELSE` Syntax Variations

- Each statement of `IF` / `ELSEIF` / `ELSE` must end with a semicolon, and the IF block must terminate with `END IF;`
  - ◊ Syntax in MySQL:

    IF search_condition THEN statement_list
      [ELSEIF search_condition THEN statement_list] ...
      [ELSE statement_list]
    END IF;

  - ◊ Example

```
DELIMITER //
CREATE FUNCTION SimpleCompare(n INT, m INT)
  RETURNS VARCHAR(20)

  BEGIN
    DECLARE s VARCHAR(20);

    IF n > m THEN SET s = '>';
    ELSEIF n = m THEN SET s = '=';
    ELSE SET s = '<';
    END IF;

    SET s = CONCAT(n, ' ', s, ' ', m);
    RETURN s;

  END //
DELIMITER ;
```

4.E.9 Repeated Execution with Loops

- Looping mechanisms:
  - ◊ `LOOP`
  - ◊ `WHILE`
  - ◊ `REPEAT`
  - ◊ `ITERATE`
  - ◊ `LEAVE`
- (We will talk a lot more about loops when we get into Python!)

Module 5

Other Database Features

Section 5.A

Database Indexes

5.A.1 What is an Index?

- An **index** is a database object comprised of a sorted list, where all the distinct values in an indexed column (or columns) are stored along with the disk address of the rows containing each value

- Indexes can help speed up data retrieval and/or updates
  - ◊ When a database does not use indexes, the SQL engine must search every row of the table to locate the desired rows (a "table scan")
  - ◊ Where data is indexed, the SQL engine can use the index to find the addresses of specific values (called a "seek")
  - ◊ Typically this is much faster than reading the entire table, especially if the table is very large (thousands or millions of records)

- For more information on how MySQL uses indexes, see:
  - ◊ https://dev.mysql.com/doc/refman/8.0/en/optimization-indexes.html

- Side note: The "proper English" plural of index is actually *indices* – however, *indexes* is more widely used in technical settings
  - ◊ However, you will sometimes see the term "indices" used instead

5.A.2 Choosing which Columns to Index

- The following types of columns are good candidates for indexing:
  - ◊ Frequently searched (`WHERE`)
  - ◊ Frequently sorted on (`ORDER BY`)
  - ◊ Frequently used to group data (`GROUP BY`)
  - ◊ Used in table joins (`JOIN`)
  - ◊ Used to calculate order statistics (`MIN` and `MAX`)

- Indexes can be created for multiple columns
  - ◊ If columns are regularly queried together, a **multiple-column index** (also called **composite index**) can be created on more than one column
    - * In MySQL, a multiple-column index can consist of up to 16 columns
    - * Multiple-column indexes can be used for queries that test all columns in the index, or just the first column, the first two columns, the first three columns, etc.

- Some columns do not need to be indexed:
  - ◊ Columns that are rarely used in queries
  - ◊ Columns that only use a limited number of distinct values (e.g. for Gender or State)
  - ◊ For small tables that do not contain a lot of data

5.A.3 Problems with Indexing

- Generally there are many advantages to indexing, but sometimes there may be disadvantages
  - ◊ Indexes are stored on disk, and the amount of space required depends on table size and number of columns in the index
  - ◊ When data is updated regularly, the database engine needs to update the indexes as well
- Because of these factors, having too many very large indexes can create write performance issues even though they solve issues with read performance
- Indexes can become fragmented and so also require periodic rebuilds / maintenance to remain effective
  - ◊ This is not a problem, exactly, just housekeeping to be mindful of!

5.A.4 Adding an Index

- In MySQL, an index can be created as part of the `CREATE TABLE` definitions, or an `CREATE INDEX` statement can be used to define a new index for an existing table
  - ◊ Implementation of indexing varies by vendor, so check documentation if using a different DBMS
- The index definition can optionally include a `UNIQUE` constraint to require that all values in the index must be distinct (though multiple `NULL` values are permitted for columns that allow `NULL`s)
- Examples:
  - ◊ Adding an index upon table creation:

```
CREATE TABLE customer (
  id INT PRIMARY KEY,
  cust_last_name CHAR(30) NOT NULL,
  cust_first_name CHAR(30) NOT NULL,
  INDEX cust_name (cust_last_name, cust_first_name)
);
```

  - ◊ Adding an index for an existing table:

```
CREATE INDEX author_name
ON author (AuthorLastName, AuthorFirstName);
```

5.A.5 Other Types of Indexes

- So far as we have been talking about indexes, we really have been talking about **secondary indexes**
  - ◊ A secondary index (sometimes called a nonclustered index) uses a logical order that differs from the actual physical stored order on disk
- Behind the scenes, a table also has a **clustered index**
  - ◊ This is a special index that stores row data and typically is synonymous with the primary key for the table
  - ◊ This clustered index can be used by a secondary index to speed up searching
    - ∗ Each record in a secondary index contains the primary key columns for the row, as well as the columns specified for the secondary index
    - ∗ This primary key value is used to locate the row via the clustered index
- **Full-text indexes** store information about significant words and their location within one or more columns of a table
  - ◊ In MySQL, a full-text index is created by adding `FULLTEXT` to the index definition for a `CHAR`, `VARCHAR`, or `TEXT` column (but cannot be used for columns with other data types)
    - ∗ For large data sets, it is much faster to load the data into a table that has no `FULLTEXT` index and then create the index afterward, rather than loading data into a table with an existing `FULLTEXT` index
- MS SQL Server also supports a unique `COLUMNSTORE` index type (columnar data format, used with data warehousing tables)

5.A.6 Dropping an Index

- An index can be removed using the DROP INDEX statement
  - ◊ Syntax:

```
DROP INDEX index_name ON table_name;
```

- While an index can be dropped at any time without affecting the table, you may find that access to previously indexed data is slower

- An index might be dropped for various reasons:
  - ◊ The index is no longer needed based on changes to the table data or index columns no longer being regularly used
  - ◊ Performance analysis indicates that the time required to maintain the index outweighs the gain to processing speed
  - ◊ The query optimizer ignores / doesn't use the index
    - * While it's beyond the scope of this course, more information about the query optimizer and indexes can be found here: https://dev.mysql.com/doc/refman/8.0/en/controlling-optimizer.html

Section 5.B

More Concepts

5.B.1 Additional Database Concepts

- You may also encounter a host of other concepts when dealing with databases:
  - ◊ Stored routines
    - ∗ Predefined SQL statements that have been compiled and stored with the database, executed using the `EXECUTE` or `CALL` keywords; used to update/modify data
    - ∗ For more detailed documentation on using stored routines, see https://dev.mysql.com/doc/refman/8.4/en/stored-routines.html
  - ◊ User-defined functions
    - ∗ Helper routines that contain one or more SQL statements, also compiled and stored with the database; do *not* modify data
  - ◊ Triggers
    - ∗ Special stored procedures that execute automatically when an insert, update, or delete operation is executed, to validate data or maintain relationships
- These concepts are beyond the scope of this course, but they are important on the administration side if you are building and managing databases

5.B.2 Types of Relational Databases

- Most modern relational databases fall into two broad categories

- Most common: **OLTP** (**Online Transactional Processing**)
  - ◊ Many, many software products utilize OLTP databases to store data
  - ◊ Enables real-time execution of very large numbers of database transactions by large numbers of people, with 24/7/365 availability
  - ◊ Typically follow very restrictive design guidelines and are very highly normalized; supports high concurrency of users and speed of updates
  - ◊ Our examples throughout the last two weeks have generally focused on OLTP databases

- Less common: **OLAP** (**Online Analytical Processing**)
  - ◊ Structured for ease of data analysis; sometimes called **data warehouses**
    - ∗ In practice, an organization might also use term “data warehouse” to refer to a set of multiple databases, and/or a data warehouse might be subdivided into “data marts” – pay attention to the context for how these terms are being used and ask questions as needed
    - ∗ For our purposes in this course, we’ll consistently use “data warehouse” to mean a single OLAP database
  - ◊ Because the focus of an OLAP database is to provide fast retrieval and readability of data, design rules are not so strict about normalization
  - ◊ Many OLAP databases are updated just once per night on a schedule, with downtime during processing

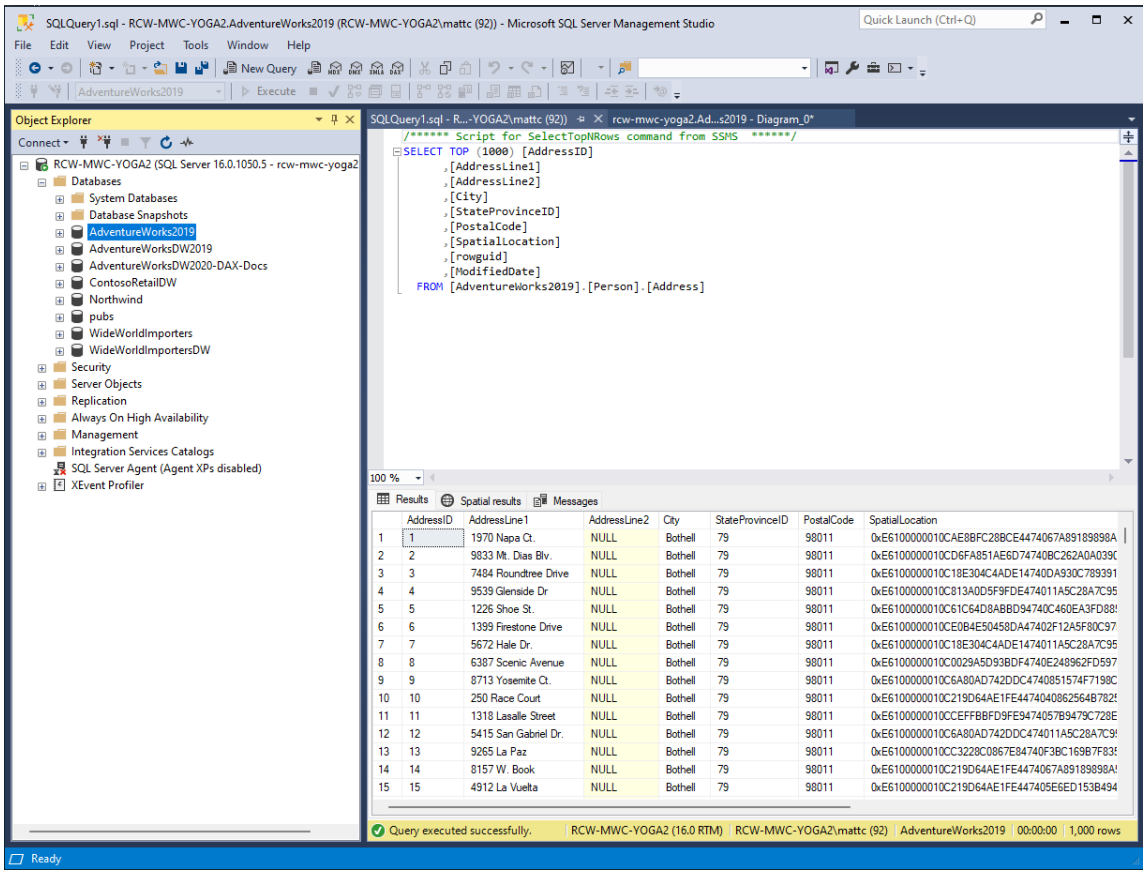
5.B.3 Many DBMS Options Exist

- As mentioned in the Module 1 introduction last week, there are numerous options for Relational Database Management Systems
  - ◊ MySQL (open source)
  - ◊ Microsoft SQL Server (commercial)
  - ◊ Oracle (commercial)
  - ◊ IBM Db2 (commercial, has community edition)
  - ◊ MariaDB (open source)
  - ◊ PostgreSQL (open source)
  - ◊ There are also other third party client software options like DBeaver (open source)
- The following pages give a quick preview of just a few of these

5.B.4 Microsoft SQL Server & SSMS

- Microsoft SQL Server Developer Edition – server software
  - ◊ This Edition is free for use for non-production purposes
  - ◊ This Edition has all of the features as the Enterprise Edition, which is much more costly but can be used for production purposes

- SQL Server Management Studio
  - ◊ This tool has been just about the same for over 20 years, proven and mature

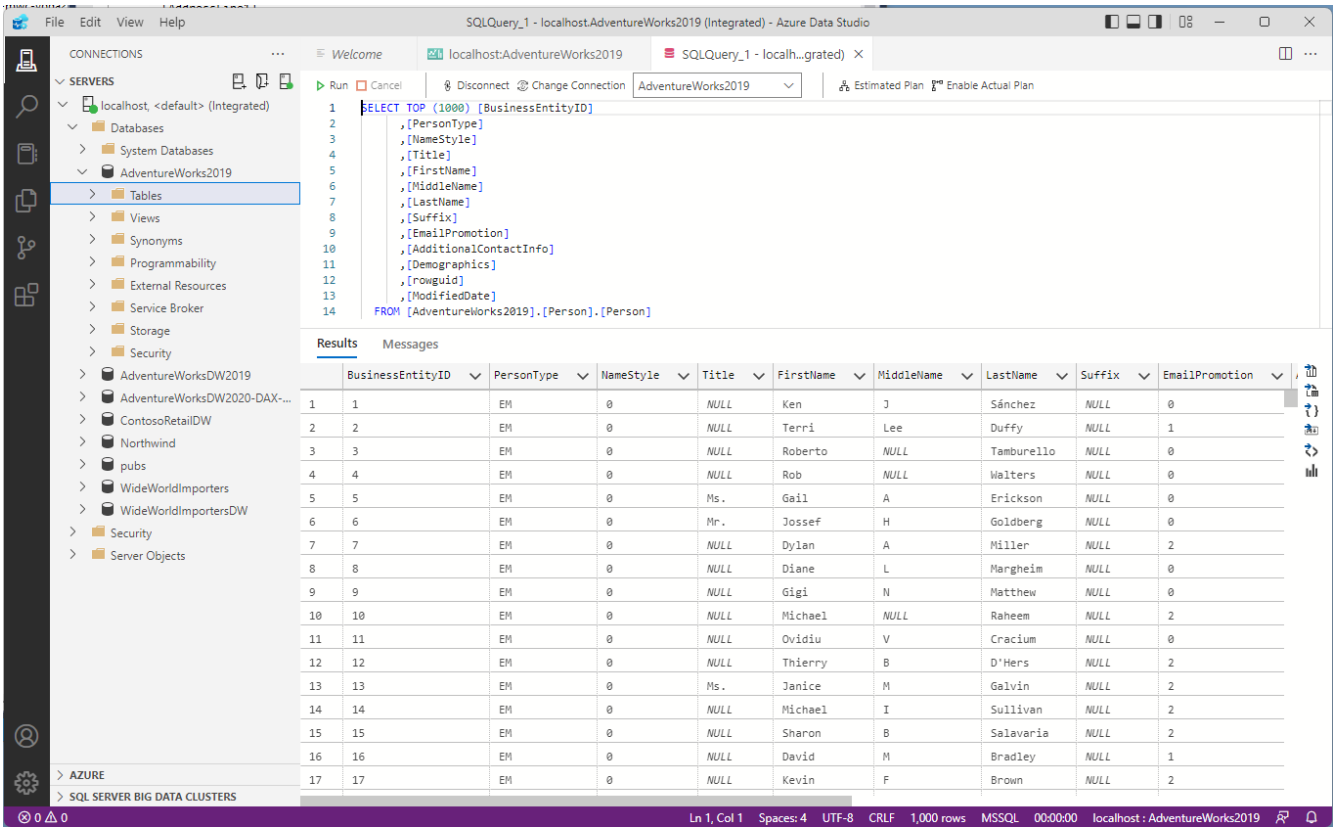


*

https://learn.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms

5.B.5 Azure Data Studio

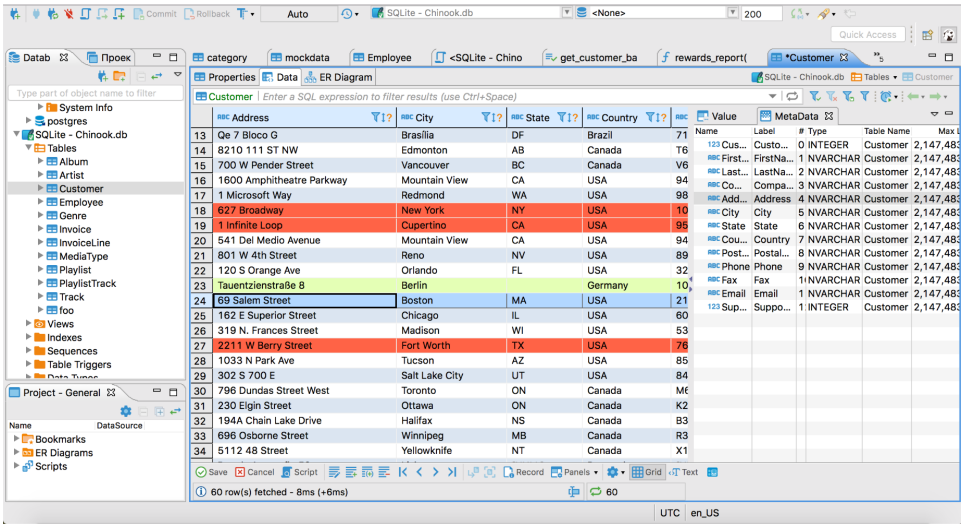
- Azure Data Studio
  - ◊ Supports Source Control
  - ◊ Supports Jupyter Notebooks
  - ◊ Supports connecting to more cloud-based services
  - ◊ Slightly more modern feel

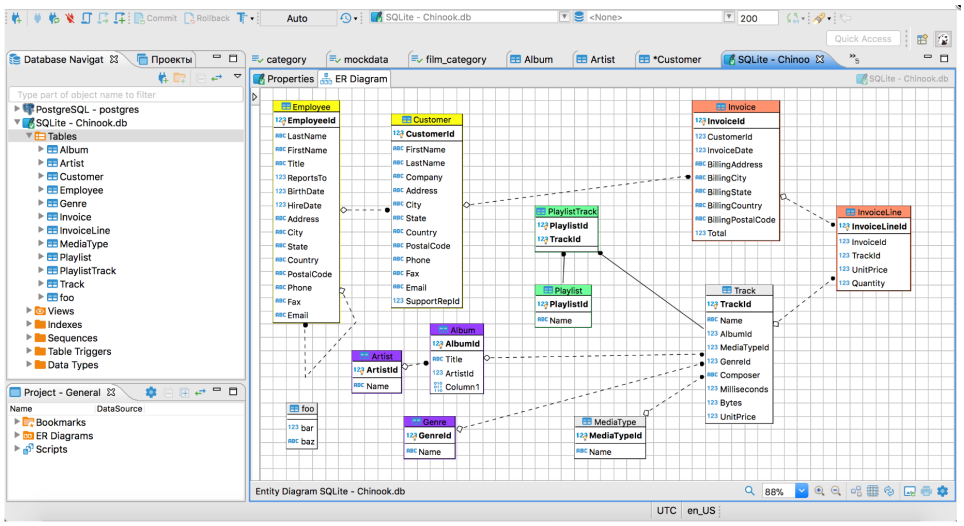


- https://azure.microsoft.com/en-us/products/data-studio/

5.B.6 3rd Party Client Software: DBeaver

- 3rd party database tools like DBeaver, HeidiSQL, and TOAD connect to databases of different types
  - ◊ Some of these also have advanced visualization or time saving features
- DBeaver is free and open source, with great visual tooling





Section 5.C

Wrap-Up

5.C.1 Review: SQL Fundamentals

- Over the past two weeks, you have covered the core skills needed to work with relational databases and SQL — here is a quick summary of what you have learned:
  - ◊ **Week 2** — the foundations: how relational databases are structured, how to design and build tables, and how to insert, update, and delete records
  - ◊ **Week 3** — putting it to work: how to write queries from simple to complex, join multiple tables, use aggregate and scalar functions, and manage complexity with views, subqueries, and CTEs
- The skills you have built are foundational for a data analyst working with structured data
  - ◊ In practice, if you work directly with databases, the majority of your day-to-day SQL work will likely center on SELECT statements for querying, filtering, joining, and aggregating data to answer business questions
  - ◊ The DDL and DML skills from Week 2 will also serve you well, even if you never need to build or modify a database structure, because now you better understand *why a database is designed the way it is* (and this will also be useful when we get to data modeling in Power BI!)

5.C.2 Best Practices

- A few habits worth carrying forward:
  - ◊ Always test your `WHERE` conditions with a `SELECT` before running an `UPDATE` or `DELETE`
  - ◊ Use aliases, comments, and consistent formatting to make your queries readable (you will thank yourself later when you come back to a query you wrote months ago!)
  - ◊ When a query starts getting very complex, consider whether a view or CTE would help to make it more manageable
  - ◊ When in doubt about behavior or syntax, consult the documentation
    - ∗ The MySQL reference manual is comprehensive and well-organized, so use it!
- SQL is a language you learn by doing
  - ◊ The concepts covered in this course will become more intuitive the more you practice writing and running actual queries
  - ◊ As you work with real data in your career, you will encounter situations not covered in this course … and that is normal! There will always be something new to learn

End of Week 3 Student Guide